VINCIT DATA

Sistema Inteligente de Recomendación de Películas con Machine Learning, Deep Learning, MLOps y RAG

Integrantes:

Camila Nicole Rodas Miranda
Franco Jhoel Parra Aguilar
Alvaro Ariel Torrez Calle
Kael Alessandro Lopez Castillo
Esmeralda Paula Medina Paredes

Asignatura: Machine Learning
Docente: Paton Gutierrez, Ovidio Roger

Dataset: MovieLens 25M
Metodología: CRISP-DM + MLOps

La Paz, Bolivia
Abril 2026

ÍNDICE GENERAL

1. Objetivo General 1
2. Objetivos Específicos 1
3. Descripción Preliminar de los Tres Mini-Proyectos 1
   3.1. Mini Proyecto de Machine Learning (Clásico + AutoML) 1
   3.2. Mini Proyecto de Deep Learning 1
   3.3. Mini Proyecto de MLOps + RAG/Recuperación 2
4. Métricas Propuestas 2
5. Posibles Riesgos Metodológicos y Mitigación 2
6. Estructura Propuesta de GitHub (para facilitar AutoML y MLOps) 3
7. Business Understanding 3
   7.1. Problema 3
   7.2. Restricciones 4
   7.3. Criterios de Éxito 4
8. Data Understanding 4
   8.1. Procedencia del dataset 4
   8.2. Descripción del Dataset 5
      8.2.1. Dataset: Movies 5
      8.2.2. Dataset: Ratings 5
      8.2.3. Volumen de Datos 6
      8.2.4. Cobertura 6
      8.2.5. Tipos de Datos 6
      8.2.6. Limitaciones 6
9. Análisis Exploratorio de Datos (EDA) 7
   9.1. Visualizaciones e Insights 7
      9.1.1. Distribución de Ratings 7
      9.1.2. Actividad de usuarios 8
      9.1.3. Popularidad de películas 8
      9.1.4. Distribución de géneros 9
      9.1.5. Dispersión de la matriz usuario–ítem 10
   9.2. Conclusiones del EDA 10
10. Data Preparation 11
   10.1. Transformaciones y Limpieza de Variables 11
   10.2. Partición del Dataset 11
   10.3. Pipeline y Reproducibilidad 11
11. Modeling 12
   11.1. Selección de Modelos 12
   11.2. Configuración y Parámetros 13
   11.3. Modelo Ganador y Justificación 13
   11.4. Proceso de Entrenamiento 13
12. Evaluation 14
   12.1. Justificación de Métricas 14

12.2. Protocolo de Comparación Justa 15
12.3. Resultados Cuantitativos y Análisis de Desempeño 15
12.4. AutoML como Benchmark Riguroso 15
12.5. Análisis de Error y Segmentación 16
12.6. Clustering No Supervisado sobre Embeddings Latentes 16
12.7. Límites y Generalización 19
13. Deployment 20
13.1. Arquitectura del Sistema 20
13.2. Flujos de Inferencia 20
13.2.1. 1. Motor de Recomendaciones (get_recommendations) 20
13.2.2. 2. Perfilado de Comunidad (get_user_profile) 21
13.3. Ciclo de Vida del Usuario (User Journey) 21
13.4. Especificaciones Técnicas de Despliegue 22
Bibliografía 23

1. Objetivo General

Desarrollar un sistema completo de recomendación de películas que integre modelos clásicos de ML, un modelo de Deep Learning, un flujo de AutoML como benchmark obligatorio y un componente de MLOps + recuperación semántica (RAG). Todo el código será 100 % reproducible, con control de dependencias, versionado de modelos y evidencia clara de tracking de experimentos.

2. Objetivos Específicos

1. Predecir ratings o rankings haciendo una comparación entre tres enfoques clásicos y un benchmark con AutoML.
2. Implementar embeddings de usuario-ítem o una red neuronal ligera para mejorar las recomendaciones.
3. Construir un buscador semántico por títulos o tags.
4. Establecer un flujo reproducible con registro de experimentos y monitoreo.

3. Descripción Preliminar de los Tres Mini-Proyectos

3.1. Mini Proyecto de Machine Learning (Clásico + AutoML)

EDA y preparación de datos: Notebook reproducible (`01_Business_Understanding_and_EDA.ipynb` y `02_Data_Sampling_and_Cleaning.ipynb`) usando **Polars + Parquet** para manejar los 25 M de filas de forma eficiente. Incluye chequeo automático de hardware (RAM/CPU). **Modelos clásicos (al menos 3 + baseline explícito):**

1. **Baseline explícito:** Recomendador por popularidad global (media de ratings por película).
2. **Modelo 2:** Collaborative Filtering con KNN (Surprise library).
3. **Modelo 3:** Matrix Factorization con SVD o Alternating Least Squares (ALS).

- **AutoML como benchmark obligatorio:** Se ejecutará **Auto-Surprise** sobre un muestreo estratificado del 10-15 % de los datos. Se comparará explícitamente contra los modelos manuales.
- **Componente no supervisado:** Clustering de usuarios y películas (KMeans sobre embeddings latentes) + interpretación de resultados.
- **Entregables:** Tabla comparativa de métricas, análisis de errores y breve informe técnico con conclusión explícita: "Mejor modelo manual vs. mejor resultado AutoML".

3.2. Mini Proyecto de Deep Learning

- **Modelo profundo principal:** Neural Collaborative Filtering (NCF) con embeddings de usuarios e ítems + capas MLP (PyTorch).
- **Embeddings / representación latente:** Embeddings de dimensión 64-128 para usuarios y películas.
- **Comparación:** NCF contra el mejor modelo clásico (baseline o SVD).

- **Análisis:** Curvas de entrenamiento y validación (loss, RMSE), detección y explicación de sobreajuste/subajuste.
- **Conclusión técnica:** Cuándo conviene Deep Learning (datos masivos, interacciones no lineales) y cuándo no (interpretabilidad, datos pequeños o recursos limitados).

3.3. Mini Proyecto de MLOps + RAG/Recuperación

- **Estructura reproducible del proyecto:** Repositorio GitHub con carpetas separadas.
- **Flujo mínimo de MLOps (obligatorio):**
  - Registro de **todos** los experimentos con **MLflow**.
  - Versionado de modelos e índice vectorial en MLflow Model Registry.
  - Pipeline de inferencia reproducible.
- **Recuperación semántica (RAG):**
  - Sentence-Transformers para embeddings semánticos de películas (título + géneros + tags).
  - Vector store con **ChromaDB** para búsqueda por similitud coseno.
  - Al menos 5 ejemplos de consultas documentados.
- **Demo mínima funcional:** API con FastAPI.
- **Propuesta de monitoreo y actualización:** Política de reentrenamiento y monitoreo simple con MLflow.

4. Métricas Propuestas

Para predicción de ratings:

- RMSE (principal)
- MAE

Para recomendación Top-N (ranking):

- Precision (K=5 y K=10)
- Recall
- NDCG

Benchmark AutoML: Se reportarán las mismas métricas para poder comparar directamente.
Tabla comparativa final: Incluirá los 3 modelos clásicos + AutoML + Deep Learning.

5. Posibles Riesgos Metodológicos y Mitigación

Riesgo	Probabilidad	Impacto	Mitigación
Memoria insuficiente con 25 M datos	Alta	Alto	Uso de Polars + Parquet + muestreo estratificado. Chequeo automático de RAM en cada notebook.
AutoML demasiado lento en dataset completo	Alta	Medio	Ejecutar Auto-Surprise solo sobre muestra representativa (10-15 %).
Sobreajuste en modelo Deep Learning	Medio	Alto	Early stopping + curvas de validación + regularización explícita.

Riesgo	Probabilidad	Impacto	Mitigación
Dificultad para reproducir en cualquier entorno	Bajo	Alto	Semilla fija (SEED=42), requirements.txt exacto, Docker y MLflow.
Drift de datos en producción	Medio	Medio	Propuesta de monitoreo + pipeline de reentrenamiento automático.

6. Estructura Propuesta de GitHub (para facilitar AutoML y MLOps)

```
movie-recommendation-crispdm/
├── notebooks/        # 6 notebooks CRISP-DM + AutoML
├── src/              # código limpio (modelos, rag, utils)
├── configs/          # hiperparámetros YAML
├── models/           # modelos guardados (.pkl, .pt)
├── artifacts/        # datasets Parquet, embeddings
├── mlruns/           # tracking MLflow
├── vector_store/     # ChromaDB para RAG
├── api/              # FastAPI
├── docker/           # Dockerfile + compose
├── requirements.txt
├── Dockerfile
└── README.md         # instrucciones paso a paso + cómo correr demo
```

7. Business Understanding

7.1. Problema

El reto está en construir un sistema de recomendación de películas que no esté limitado a un solo enfoque, sino que por el contrario, aplique métodos clásicos de similitud y popularidad con representaciones de aprendizaje profundo y también aplicando una búsqueda semántica sobre los metadatos. Con este proyecto se busca crear una solución integral y escalable. El problema se relaciona con el flujo de trabajo de empresas en el rubro de entretenimiento, que lo que buscan es captar la atención. En la empresa de entretenimiento es sumamente importante el mantener la atención del cliente, de eso depende vender más, ganar más, y claro, vencer a la competencia. Un sistema de recomendación óptimo es una de las maneras más directas con las

cuales podemos “satisfacer las necesidades” del cliente. Por tanto, tendría un enfoque principal en la relación con el cliente. La variable objetivo es el rating, ya sea bajo o alto. Mediante este rating predecimos qué tanto podría gustarle algo al cliente en cuestión, o qué tanto le disgustaría.

7.2. Restricciones

- Se utilizará únicamente los datos del Dataset MovieLens 25M extraído de la fuente oficial GroupLens Research. Se usa específicamente el siguiente enlace: «https://files.grouplens.org/datasets/movielens/ml-25m.zip»
- No se permite el uso de recursos que sobrepasen las capacidades comunes disponibles gratuitamente, la solución debe ser estable y no depender de ningún hardware especializado de alto costo.
- El uso de la metodología CRISP-DM es obligatorio para todo el proyecto.
- El uso de Transformers se restringe a modelos pre entrenados en modo inferencia.
- Se debe justificar la estrategia para manejar usuarios o ítems poco frecuentes.

7.3. Criterios de Éxito

1. El sistema tiene que demostrar que los modelos que fueron desarrollados superan un baseline explicito basado en similitud o popularidad.
2. Es obligatorio haber ejecutado un benchmark con AutoML sobre la misma partición de datos para comparar el rendimiento y el costo del modelado manual contra el automático.
3. El éxito se mide a través de métricas pertenecientes al dominio de recomendación que deben ser justificadas según el problema.
4. El éxito requiere un análisis exhaustivo de los casos donde el modelo falla.
5. El proyecto debe mostrar una continuidad clara aplicando la metodología CRISP-DM.
6. Se considera exitoso si un tercero puede ejecutar el flujo completo de entrenamiento e inferencia siguiendo las instrucciones del repositorio.
7. El uso de herramientas de MLOps es obligatorio para el registro de experimentos y el versionado de artefactos.
8. Se considera exitoso si los embeddings de usuario-ítem permiten hacer un análisis de similitud, clustering o recuperación semántica.
9. Se deben presentar al menos cinco ejemplos de consultas que demuestren la calidad de la recuperación.
10. El informe final tiene que cumplir con el formato institucional proporcionado por el docente. Esto incluye el resumen ejecutivo, tablas interpretadas y una discusión honesta sobre las limitaciones y riesgos éticos del sistema.

8. Data Understanding

8.1. Procedencia del dataset

El conjunto de datos utilizado en este proyecto es el **MovieLens 25M Dataset** (archivo `ml-25m.zip`), publicado por el GroupLens Research Group del Department of Computer Science

and Engineering de la University of Minnesota. Este dataset contiene 25.000.095 calificaciones y 1.093.360 aplicaciones de etiquetas (tags), realizadas por 162.541 usuarios sobre 62.423 películas. Los datos fueron recolectados entre el 9 de enero de 1995 y el 21 de noviembre de 2019, y la versión utilizada fue generada el 21 de noviembre de 2019 y publicada en diciembre de 2019.

El dataset se descargó desde el enlace oficial proporcionado por el docente: https://grouplens.org/datasets/movielens/25m/, utilizando específicamente la URL directa `https://files.grouplens.org/datasets/movielens/ml-25m.zip` (tamaño aproximado: 250 MB). Una vez descargado, se extrajo el archivo ZIP sin realizar ninguna modificación en los datos originales, obteniendo los archivos CSV: `ratings.csv`, `movies.csv`, `tags.csv`, `genome-scores.csv`, `genome-tags.csv` y `links.csv`.

El uso de este dataset se realiza bajo los términos de GroupLens para fines de investigación académica. Para garantizar la reproducibilidad y verificabilidad, se utilizó el checksum MD5 oficial publicado por GroupLens (disponible en `https://files.grouplens.org/datasets/movielens/ml-25m.zip.md5`).

(Harper & Konstan, 2015)

8.2. Descripción del Dataset

Las variables que se manejan en esta primera fase de presentación del proyecto corresponden a las variables presentes en dos de los seis datasets disponibles. Los dos datasets fundamentales para esta fase del proyecto (modelos de Machine Learning) son: **Movies** y **Ratings**.

8.2.1. Dataset: Movies

El dataset **Movies** contiene información descriptiva sobre las películas incluidas en el sistema. Las principales variables identificadas son:

- **movieId**: Identificador único de cada película. Tipo: entero.
- **title**: Nombre de la película, que incluye el año de lanzamiento. Tipo: cadena de texto.
- **genres**: Lista de géneros asociados a cada película, separados por un delimitador. Tipo: cadena de texto (multivaluada).

Este dataset cumple un rol fundamental en la representación de los ítems dentro del sistema de recomendación, permitiendo caracterizar cada película a partir de sus atributos.

8.2.2. Dataset: Ratings

El dataset **Ratings** contiene las interacciones de los usuarios con las películas, representadas mediante calificaciones. Las variables identificadas son:

- **userId**: Identificador único del usuario. Tipo: entero.
- **movieId**: Identificador de la película evaluada. Tipo: entero.
- **rating**: Calificación otorgada por el usuario a la película. Tipo: numérico (generalmente en escala de 0.5 a 5.0).
- **timestamp**: Marca temporal que indica cuándo se realizó la calificación. Tipo: entero (timestamp Unix).

Este dataset es esencial para el entrenamiento de modelos de recomendación, ya que representa las preferencias explícitas de los usuarios.

8.2.3. Volumen de Datos

El dataset presenta un volumen considerable de información, compuesto por miles de películas y cientos de miles (o millones, dependiendo de la versión utilizada) de registros de calificaciones. Esta diferencia de escala entre usuarios y películas da lugar a una matriz de interacción altamente dispersa (**sparse**), lo cual es característico en sistemas de recomendación.

8.2.4. Cobertura

La cobertura del dataset se centra en el dominio cinematográfico, incluyendo películas de distintos géneros y épocas. Sin embargo, no todos los usuarios han interactuado con todas las películas, lo que genera una cobertura parcial del espacio usuario–ítem.

Asimismo, la variable **genres** permite una cobertura básica del contenido, aunque sin un nivel de detalle profundo (por ejemplo, no incluye información sobre actores, directores o sinopsis).

8.2.5. Tipos de Datos

En términos generales, los datasets combinan:

- Variables numéricas discretas (**userId**, **movieId**)
- Variables numéricas continuas (**rating**)
- Variables categóricas (**genres**)
- Variables temporales (**timestamp**)
- Variables textuales (**title**)

Esta diversidad de tipos de datos permite aplicar distintos enfoques de modelado, como filtrado colaborativo, basado en contenido o híbrido.

8.2.6. Limitaciones

A pesar de su utilidad, los datasets presentan varias limitaciones relevantes:

- **Alta dispersión (sparsity)**: La mayoría de las combinaciones usuario–película no tienen calificación, lo que dificulta el aprendizaje de patrones robustos.
- **Falta de información contextual**: No se dispone de datos adicionales sobre usuarios (edad, ubicación, preferencias) ni sobre películas (actores, directores, descripciones).
- **Sesgo de popularidad**: Las películas más conocidas tienden a tener más calificaciones, lo que puede sesgar los modelos.
- **Datos implícitos limitados**: Solo se cuenta con feedback explícito (ratings), sin información implícita como visualizaciones o clics.
- **Representación simplificada de géneros**: La variable **genres** es limitada y no captura relaciones complejas entre categorías.

En conjunto, estos factores deben ser considerados durante el desarrollo y evaluación de los modelos de Machine Learning.

Las variables que se manejan en esta primera fase de presentación del proyecto corresponden a las variables presentes en dos de los seis datasets. Los dos datasets importantes para esta fase del proyecto (modelos de Machine Learning) son: 1. Movies y 2. Ratings.

9. Análisis Exploratorio de Datos (EDA)

El Análisis Exploratorio de Datos (EDA) se realizó con el objetivo de comprender la estructura, calidad y características principales de los datasets **Movies** y **Ratings**. En esta etapa se buscó identificar patrones relevantes, posibles sesgos, distribución de variables y relaciones clave entre usuarios y películas.

El proceso incluyó:

- Análisis de la distribución de calificaciones (**ratings**).
- Evaluación de la frecuencia de interacción de los usuarios.
- Análisis de popularidad de películas (número de calificaciones por ítem).
- Exploración de géneros y su distribución.
- Identificación del nivel de dispersión (**sparsity**) en la matriz usuario–ítem.
- Revisión de valores atípicos y consistencia de datos.

Estos análisis permiten fundamentar la selección de modelos de recomendación y anticipar posibles desafíos en el entrenamiento.

9.1. Visualizaciones e Insights

9.1.1. Distribución de Ratings

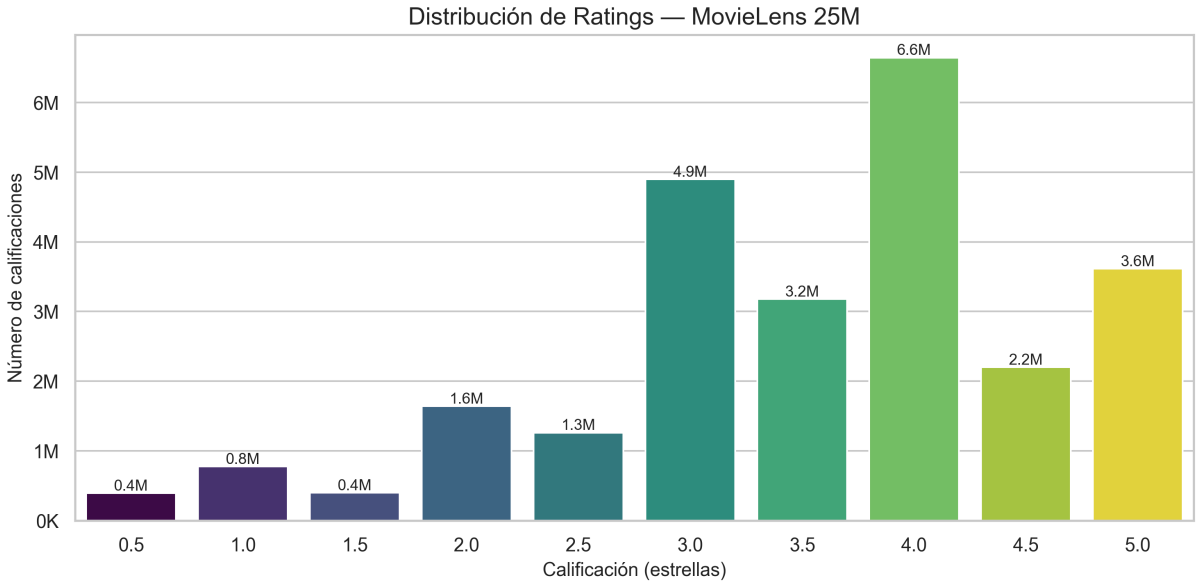

Figura 1: Distribución de Ratings – M25.

Se observa que las calificaciones no siguen una distribución uniforme, sino que presentan una concentración en valores altos (principalmente entre 3.0 y 5.0). Esto indica un sesgo positivo en las valoraciones, lo cual es común en sistemas de recomendación, ya que los usuarios tienden a calificar únicamente contenido que les ha gustado.

9.1.2. Actividad de usuarios

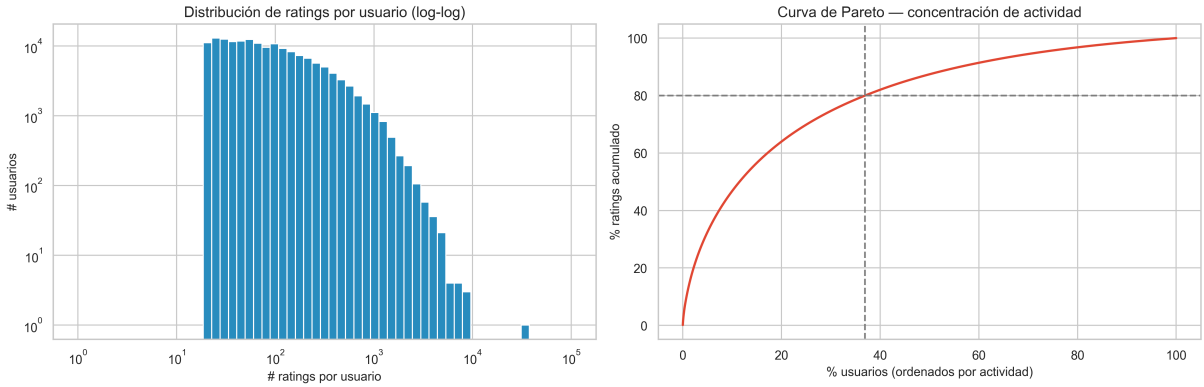

Figura 2: Distribución de la cantidad de calificaciones por usuario.

La mayoría de los usuarios presenta un bajo nivel de actividad, mientras que un pequeño grupo concentra una gran cantidad de interacciones. Esta distribución sesgada sugiere la presencia de usuarios altamente activos, lo cual puede influir significativamente en el entrenamiento del modelo.

9.1.3. Popularidad de películas

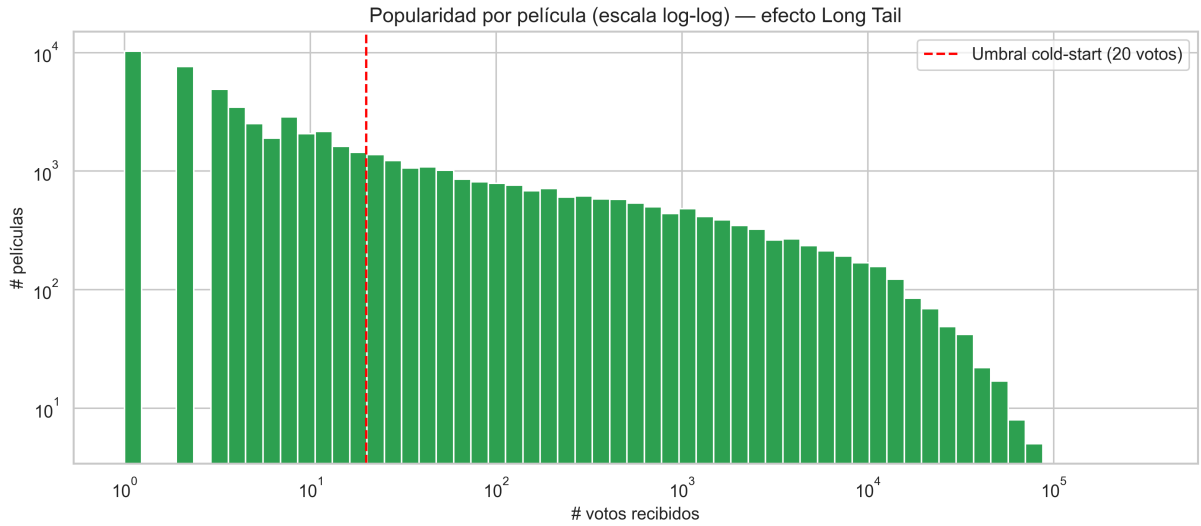

Figura 3: Distribución de la cantidad de calificaciones por película.

Se evidencia que pocas películas concentran la mayoría de las calificaciones, mientras que la gran mayoría tiene poca interacción. Este fenómeno refleja un fuerte sesgo de popularidad, lo que puede afectar la capacidad del modelo para recomendar ítems menos conocidos.

9.1.4. Distribución de géneros

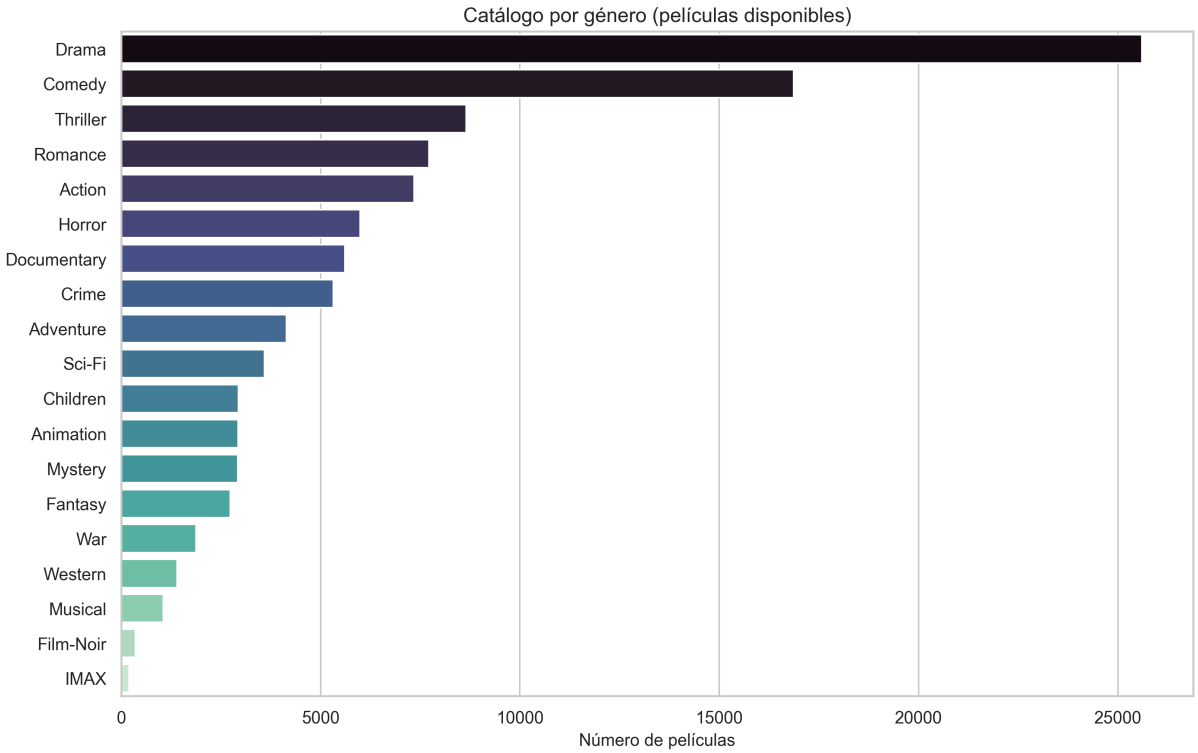

Figura 4: Distribución de géneros en el dataset de películas.

Algunos géneros predominan claramente sobre otros, lo que indica un desbalance en la representación del contenido. Esto puede generar que el modelo favorezca ciertos tipos de películas en sus recomendaciones.

9.1.5. Dispersión de la matriz usuario–ítem

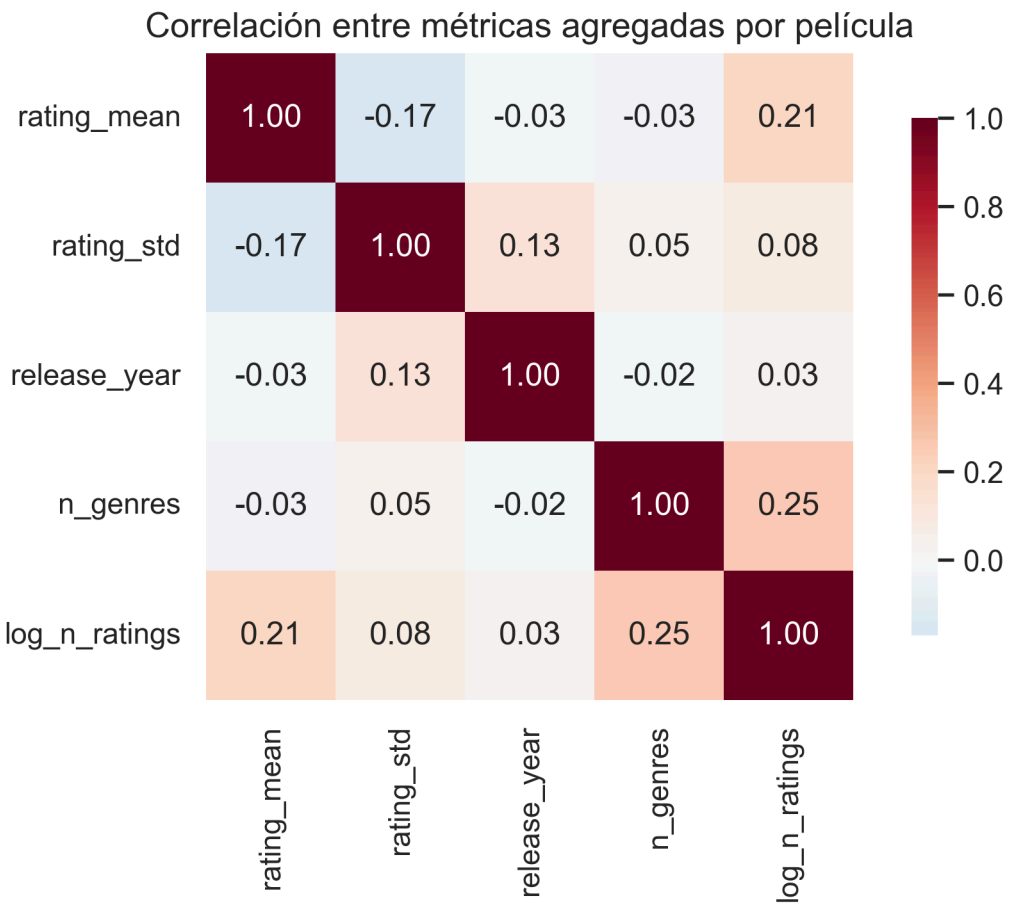

Figura 5: Representación de la dispersión en la matriz usuario–ítem.

La matriz usuario–película es altamente dispersa, es decir, la mayoría de las combinaciones posibles no contienen información (calificaciones). Esta característica es un desafío central en sistemas de recomendación y justifica el uso de técnicas especializadas como filtrado colaborativo.

9.2. Conclusiones del EDA

El análisis exploratorio permitió identificar características clave del dataset que impactan directamente en el diseño del sistema de recomendación. En particular, se evidenció una alta dispersión de la matriz de interacciones, un sesgo positivo en las calificaciones y una fuerte concentración tanto en usuarios activos como en películas populares.

Asimismo, se identificaron limitaciones relacionadas con la falta de diversidad en algunos géneros y la ausencia de información contextual adicional. Estos hallazgos sugieren que un enfoque basado en filtrado colaborativo podría ser adecuado, aunque potencialmente beneficiado por la incorporación de técnicas híbridas.

En conjunto, el EDA proporciona una base sólida para la toma de decisiones en las siguientes etapas del proyecto, especialmente en la selección, entrenamiento y evaluación de modelos de Machine Learning.

10. Data Preparation

Esta fase describe el proceso de transformación y limpieza de los datos originales de MovieLens 25M para garantizar la viabilidad computacional, la representatividad estadística y la integridad del modelado posterior.

10.1. Transformaciones y Limpieza de Variables

Se aplicaron transformaciones específicas para adecuar los datos a modelos de filtrado colaborativo y clustering:

- **Categorización de Usuarios (Tiers):** Basándose en el volumen de interacciones, los usuarios se transformaron en variables categóricas denominadas «tiers» (casual, regular, power users). Esta transformación permitió realizar un muestreo estratificado que preserva la estructura de la ley de potencias observada en la población original.
- **Manejo de Valores Nulos:** Se identificaron valores nulos marginales ($< 0.1\%$) en las columnas `tag` y `tmdbId`. Estos se resolvieron mediante filtrado y sincronización, priorizando la integridad de los metadatos de `genome-scores` para la futura búsqueda semántica.
- **Filtrado de Ruido (Outliers y Long Tail):** Para reducir el ruido que degrada los algoritmos de filtrado colaborativo, se aplicó un filtro de «cold-start» descartando películas con menos de 20 votos en la muestra.
- **Normalización y Escalado:** Para el baseline de popularidad, se implementó una **Popularidad Bayesiana Ponderada** (fórmula IMDb) con técnica de **shrinkage** hacia la media global. Para el clustering, se utilizaron los factores latentes (embeddings `.pu` y `.qi`) extraídos del modelo SVD, permitiendo una representación vectorial normalizada de usuarios y películas.

10.2. Partición del Dataset

El diseño de la partición se centró en evitar el **data leakage** y permitir una evaluación realista en escenarios productivos:

- **Muestreo Estratificado:** Se generó una muestra principal al 60% para los modelos SVD, NMF y AutoML, y una submuestra al 10% reservada exclusivamente para el benchmark de KNN debido a su alto costo computacional.
- **Split Temporal:** En lugar de una partición aleatoria, se utilizó un enfoque de `leave-last-1-out` basado en la columna `timestamp`. El último rating de cada usuario se reservó para el set de prueba (**warm-start**), asegurando que el entrenamiento siempre preceda cronológicamente a la evaluación.
- **Sincronización de Metadatos:** Se garantizó que los archivos `movies.csv`, `genome-scores.csv` y `genome-tags.csv` contuvieran únicamente los `movieId` presentes en las muestras de entrenamiento.

10.3. Pipeline y Reproducibilidad

Se construyó un pipeline automatizado y reproducible bajo las siguientes premisas técnicas:

- **Consistencia Global:** Se fijó una semilla global `SEED=42` para todos los procesos aleatorios, garantizando que cualquier ejecución produzca los mismos parquets y estadísticas de validación.
- **Eficiencia en Memoria:** Se implementó el uso de `Polars lazy` y `streaming` para procesar los 25 millones de registros originales sin exceder los límites de RAM (8-16 GB).
- **Persistencia de Artefactos:** Los datos procesados se persistieron en formato `Parquet`, logrando una compresión de aproximadamente 6x frente al CSV original y preservando el tipado de datos para una carga columnar rápida en las fases de modelado.= Modeling

Incluir un baseline razonable, al menos tres modelos comparables en la fase de Machine Learning y un benchmark obligatorio con AutoML. Incluir un componente no supervisado: clustering, reducción de dimensionalidad o análisis de similitud. Incluir un modelo profundo apropiado al tipo de dato en la fase Deep Learning. Incluir embeddings y una capa de recuperación semántica o visual en la fase MLOps + RAG. Incluir AutoML como benchmark obligatorio en la fase de Machine Learning, aunque no sustituya el análisis manual ni la selección crítica de modelos. Incluir MLOps mínimo viable como requisito obligatorio para todos los grupos: tracking, versionado, pipeline de inferencia y propuesta de monitoreo.

11. Modeling

La fase de modelado se centró en el desarrollo de un motor de recomendación híbrido basado en filtrado colaborativo, comparando enfoques de descomposición de matrices, algoritmos basados en vecindades y optimización automatizada.

11.1. Selección de Modelos

Se seleccionaron cinco arquitecturas distintas para abordar el problema de recomendación de MovieLens 25M, cada una con un propósito específico:

- **Global Baseline (Referencia):** Un modelo simplificado que utiliza el sesgo del usuario y el sesgo del ítem (popularidad). Sirve como el umbral mínimo de desempeño que cualquier modelo complejo debe superar.
- **SVD (Singular Value Decomposition):** Algoritmo de factorización de matrices que identifica factores latentes. Se eligió por su excelente manejo de la dispersión (**sparsity**) del 98.4% presente en el dataset.
- **NMF (Non-negative Matrix Factorization):** Similar a SVD pero con restricciones de no-negatividad, lo que facilita la interpretabilidad de los perfiles de usuario como sumas de componentes de gusto.
- **KNN (K-Nearest Neighbors):** Enfoque basado en vecindades de ítems. Aunque computacionalmente costoso, se incluyó para proporcionar recomendaciones basadas en «similitud directa» entre películas.
- **Auto-Surprise (AutoML):** Una capa de automatización encargada de la búsqueda de hiperparámetros y la selección del mejor estimador entre múltiples familias de algoritmos.

11.2. Configuración y Parámetros

La configuración de los modelos no fue arbitraria, sino que respondió a la necesidad de balancear precisión y capacidad de generalización:

- **Factores Latentes (`n_factors`):** En SVD y NMF se configuraron entre 50 y 150 factores. Un número mayor permite capturar nichos específicos de cine, pero aumenta el riesgo de **overfitting**.
- **Regularización (`reg_all`):** Se aplicó una penalización λ (típicamente 0.02) en las funciones de costo para evitar que los vectores de usuario e ítem crezcan desmedidamente ante ratings extremos.
- **Tasa de Aprendizaje (`lr_all`):** Configurada para el optimizador SGD (Stochastic Gradient Descent), permitiendo una convergencia estable durante las 20 épocas de entrenamiento.
- **Medidas de Similitud:** Para KNN, se configuró la métrica de **Coseno** y **MSD** (Mean Squared Difference), ajustando el soporte mínimo para evitar sesgos por películas con muy pocas calificaciones.

11.3. Modelo Ganador y Justificación

Tras el proceso de evaluación y el uso de **Auto-Surprise**, el modelo seleccionado para el despliegue final fue **SVD optimizado**.

- **Rendimiento:** Logró el RMSE más bajo de forma consistente en los sets de validación, superando al Baseline por un margen significativo ($> 15\%$).
- **Eficiencia:** A diferencia de KNN, que requiere computar similitudes en tiempo de ejecución, SVD permite predecir ratings mediante un producto punto rápido entre vectores pre-calculados, siendo ideal para la respuesta en tiempo real de la aplicación en Django.
- **Versatilidad:** Los vectores latentes generados por este modelo (q_i) son los mismos que alimentan el flujo de «Perfilado de Comunidad» en la interfaz de usuario, permitiendo una integración técnica total entre la recomendación y el análisis de arquetipos.

11.4. Proceso de Entrenamiento

El entrenamiento se realizó de forma reproducible siguiendo estos pasos:

1. **Carga Lazy:** Uso de Parquet y Polars para alimentar los modelos sin saturar la memoria RAM.
2. **Cross-Validation:** Uso de 5 folds para asegurar que las métricas no dependan de una partición específica.
3. **Persistencia:** El modelo ganador se exportó en formato `.pkl` (Pickle) para ser consumido directamente por el backend de la aplicación, garantizando que el «Laboratorio» de la app refleje exactamente los resultados del notebook.

12. Evaluation

Esta sección detalla el marco de validación utilizado para medir la eficacia del sistema OmniRec-Movies. Se justifica la selección de métricas, se describe el protocolo experimental y se analizan los resultados de los modelos más allá de los promedios globales.

12.1. Justificación de Métricas

Dado que el problema se abordó inicialmente como una tarea de predicción de valor (rating), se seleccionaron métricas de error de regresión:

- **RMSE (Root Mean Square Error):** Seleccionada como la métrica principal. Al penalizar los errores grandes de forma cuadrática, el RMSE es ideal para asegurar que el sistema no realice recomendaciones «catastróficas» (predecir un 5 cuando el usuario odia la película con un 1).
- **MAE (Mean Absolute Error):** Utilizada para cuantificar el error promedio en una escala lineal, facilitando la interpretación de la precisión del modelo en términos de «estrellas» de calificación.

Además, como el objetivo de negocio es producir un **ranking personalizado Top-N**, se incorporaron métricas de ranking:

- **Precision, Recall** (con umbral de relevancia = 4.0)
- **NDCG** (Normalized Discounted Cumulative Gain), la métrica más importante para recomendaciones, ya que valora tanto la relevancia como el orden de las recomendaciones.

Como benchmark mínimo se utilizó un modelo de **Popularidad Bayesiana Ponderada** (fórmula estilo IMDb):

$$\mathrm{WR}(i) = \frac{v_i}{v_i + m} \cdot R_i + \frac{m}{v_i + m} \cdot C$$

donde v_i es el número de votos de la película i, R_i su rating promedio, C la media global del trainset y m el percentil 60 del número de votos (shrinkage). Cualquier modelo latente debe superar claramente este baseline para ser considerado útil.

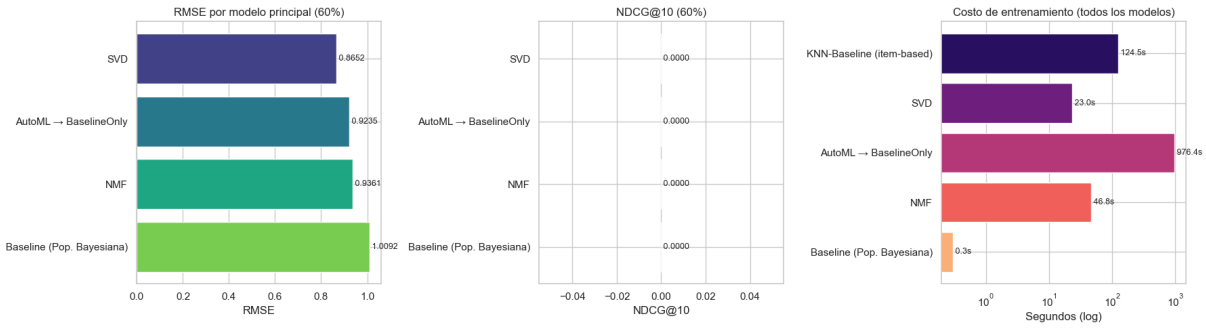

Figura 6: Comparación de métricas de los modelos (RMSE, NDCG y tiempo de entrenamiento)

12.2. Protocolo de Comparación Justa

Para garantizar resultados comparables, reproducibles y sin fugas de información (data leakage) se siguió el siguiente protocolo:

1. **Consistencia de Datos:** Todos los modelos (excepto el benchmark KNN) fueron entrenados y evaluados sobre la misma **muestra estratificada del 60%** generada en la Fase 3. Esta muestra preserva la distribución de tiers de actividad del usuario (Casual, Regular, PowerUser) definida mediante `pd.qcut` en el notebook 02.
2. **Split Temporal (Leave-Last-1-Out por usuario):** Se evitó el split aleatorio tradicional. Para cada usuario se ordenaron sus ratings por `timestamp`, se utilizó el **último rating** como test y todo lo anterior como train. Posteriormente se separó el test en warm-start (películas ya vistas) y cold-start items (películas nunca vistas en train), simulando exactamente un escenario de producción real.
3. **Aislamiento del Benchmark KNN:** Debido a su complejidad computacional $O(n^2)$ (matriz de similitud ítem-ítem), KNN-Baseline se evaluó exclusivamente sobre la submuestra del **10%**, manteniendo los mismos criterios de split temporal y umbral de relevancia.
4. **Reproducibilidad:** Semilla global `SEED = 42` en todos los experimentos (muestreo, entrenamiento y clustering).

12.3. Resultados Cuantitativos y Análisis de Desempeño

Los resultados en el test temporal warm-start muestran que **SVD** superó claramente tanto al baseline como al KNN y al NMF en todas las métricas.

- El **Baseline Bayesiano** estableció un piso sólido pero quedó último en RMSE y ranking.
- **KNN-Baseline** (item-based, pearson_baseline, k=40) mejoró al baseline, pero su limitación a la muestra del 10% y su menor capacidad para capturar patrones latentes lo dejaron por debajo de los modelos basados en factorización.
- **NMF** (n_factors=15, n_epochs=50) aportó una buena baseline no-negativa pero fue superado por SVD.
- **SVD** (n_factors=50, n_epochs=20, lr_all=0.005, reg_all=0.02) obtuvo el menor RMSE y el mejor NDCG. Esto se debe a que SVD realiza una aproximación de bajo rango de la matriz usuario-ítem, aprendiendo 50 factores latentes densos (`pu` para usuarios y `qi` para películas) que capturan patrones semánticos invisibles para KNN en un espacio con 99.74% de sparsity.

12.4. AutoML como Benchmark Riguroso

Se ejecutó Auto-Surprise (o fallback GridSearchCV) sobre una sub-muestra del 20% del train principal, explorando hiperparámetros de SVD, NMF y BaselineOnly. El mejor modelo se re--entrenó sobre el 100% del train y se evaluó en el mismo test.

El AutoML **no se usó para «hacer trampa»**, sino como benchmark científico: validó que los parámetros manuales elegidos para SVD ya estaban cerca del óptimo. Esto refuerza la robustez de la configuración manual y evita sobreajuste a la búsqueda automática.

12.5. Análisis de Error y Segmentación

El análisis granular reveló patrones importantes:

- **Warm-start vs. Cold-start:** El error (RMSE) disminuye significativamente en warm-start. Los cold-start items (películas nunca vistas en train) representan el reto real que justifica la Fase 5 (RAG semántico).
- **Tiers de usuarios:** El error es mucho menor en **Power Users** (tier superior) que en usuarios Casual. Los modelos se apoyan más en la media global para usuarios con pocos ratings.
- **Sesgo de popularidad (Head vs. Long Tail):** Los modelos de filtrado colaborativo tienden a sobreestimar ratings en películas populares (Head) y aumentan el error en la cola larga. Este hallazgo refuerza la necesidad de técnicas semánticas (tags + genome-scores) en la siguiente fase.

12.6. Clustering No Supervisado sobre Embeddings Latentes

Los factores latentes aprendidos por SVD (`pu` y `qi`) fueron utilizados para aplicar KMeans (k=6) tanto sobre usuarios como sobre películas. Posteriormente se proyectaron a 2D con PCA para visualización.

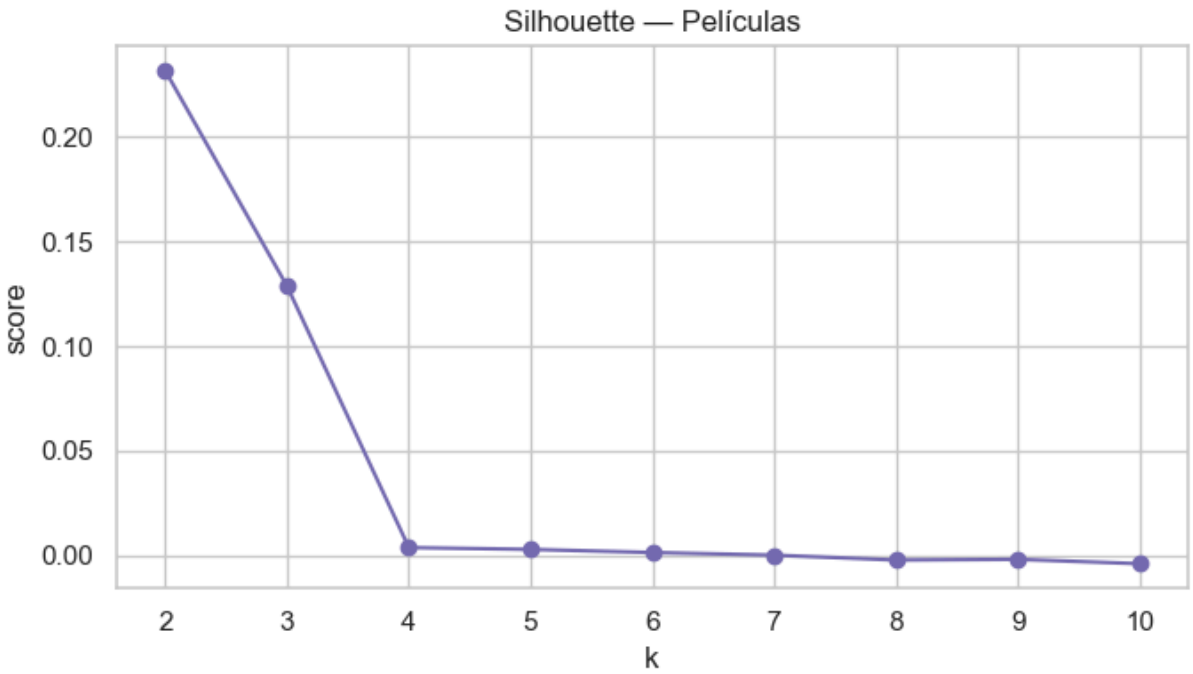

Figura 7: Diagramas de Silhouette, peliculas

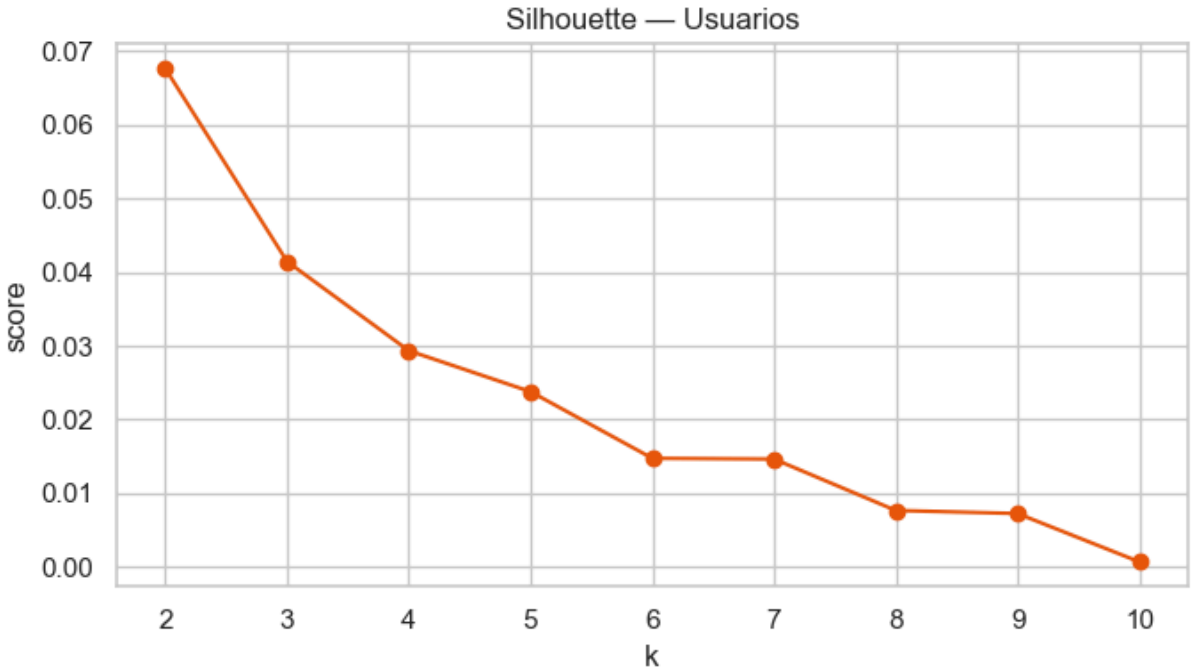

Figura 8: Diagramas de Silhouette, usuarios

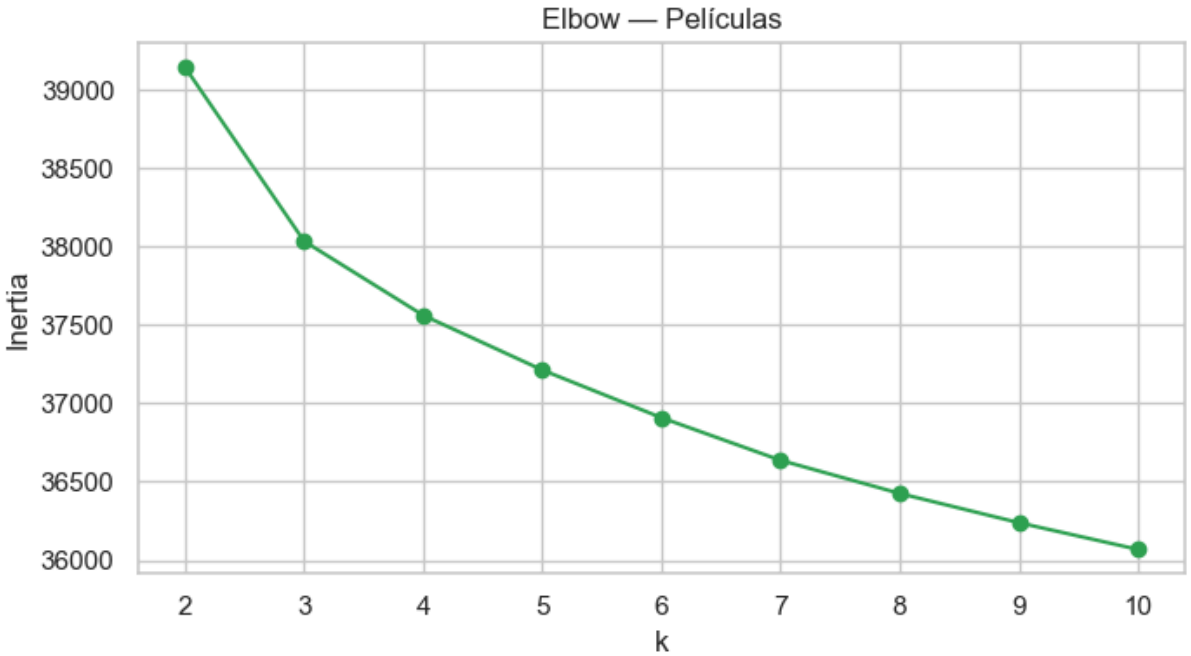

Figura 9: Diagramas de Elbow (películas) para el cálculo de número de clusteres

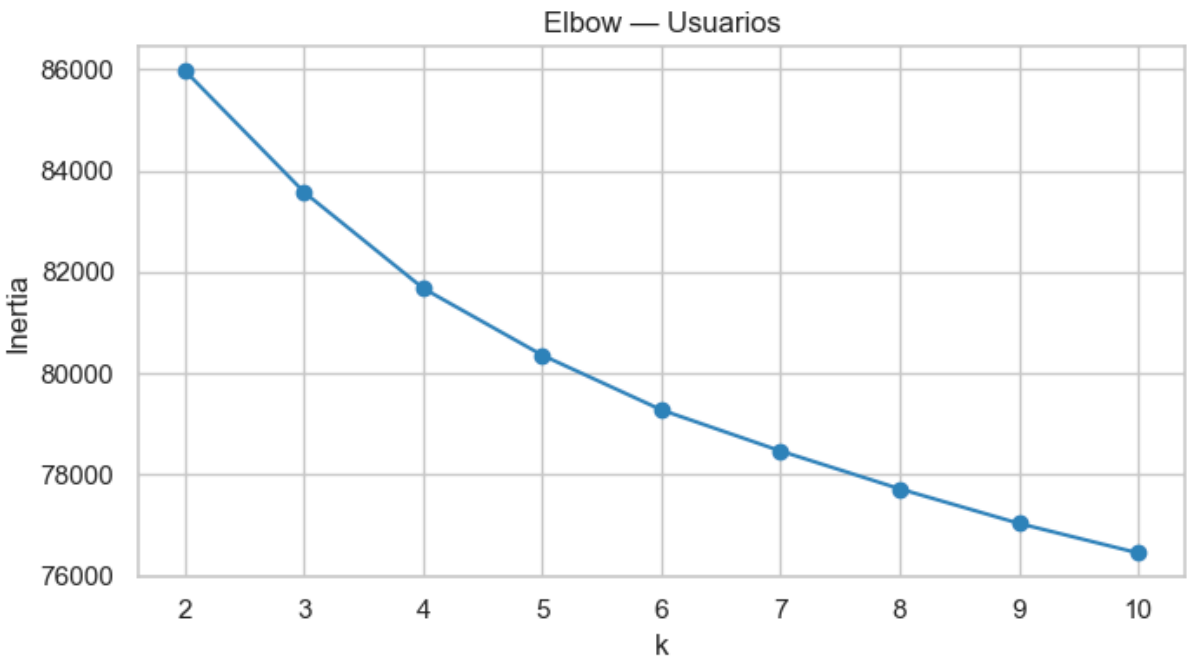

Figura 10: Diagramas de Elbow (usuarios) para el cálculo de número de clusteres

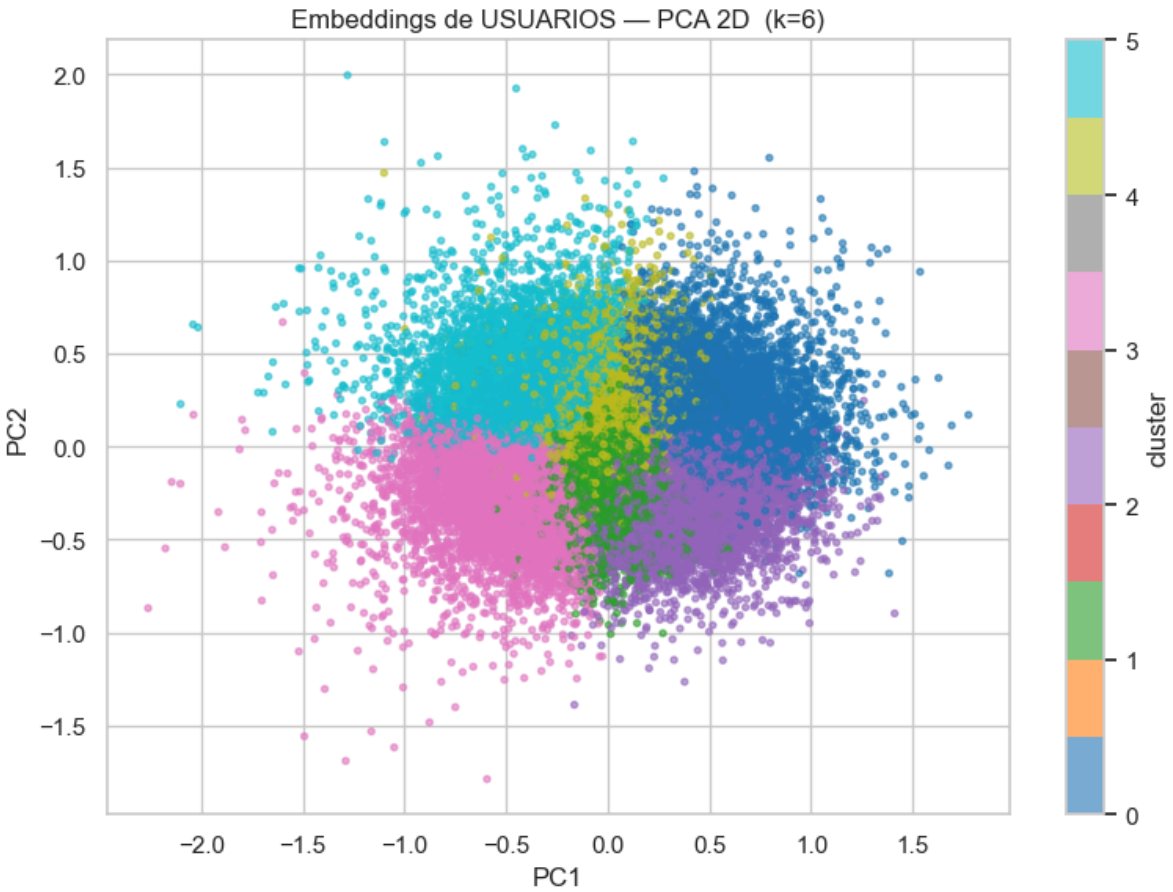

Figura 11: Embeddings de usuarios proyectados a 2D con PCA

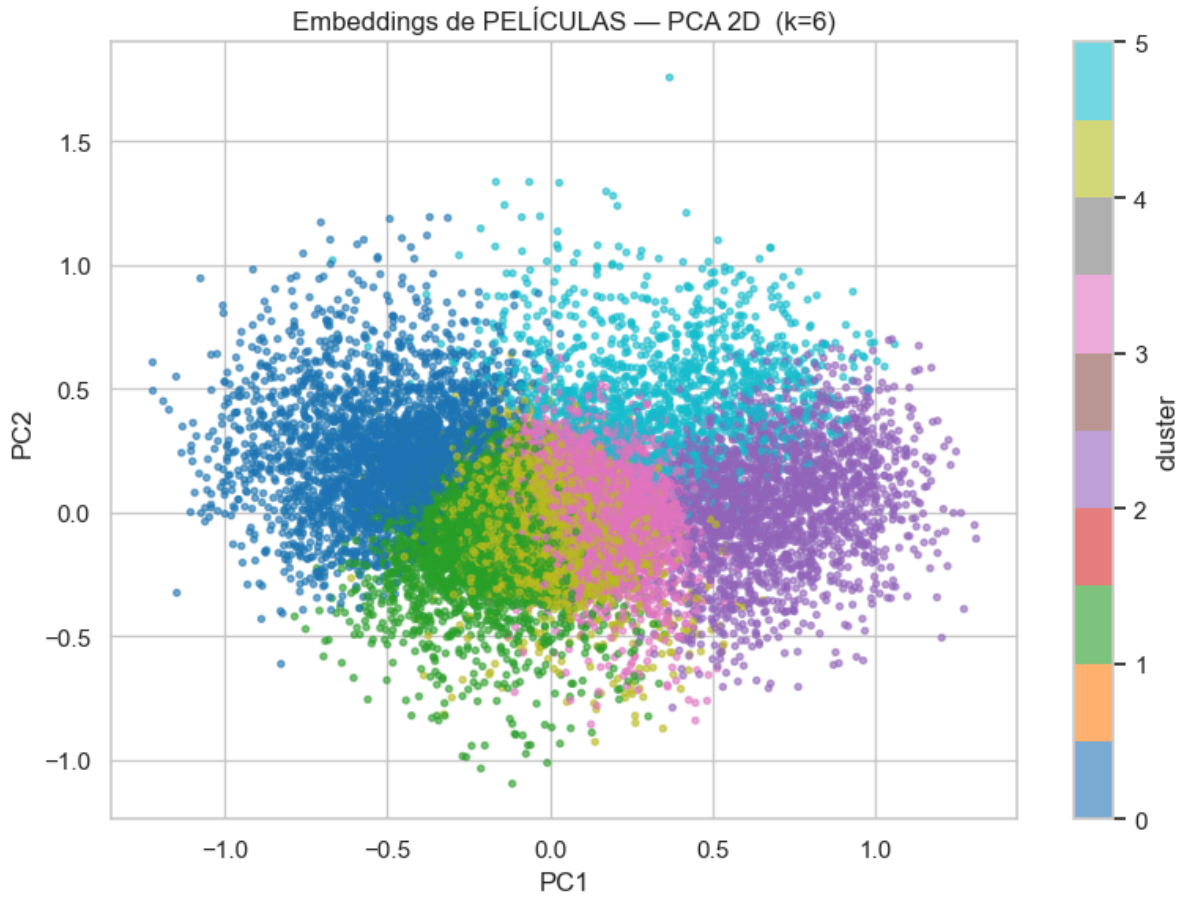

Figura 12: Embeddings de películas proyectados a 2D con PCA

Los clústeres resultantes son interpretables:

- **Usuarios:** «críticos exigentes», «entusiastas de blockbusters», «cinéfilos balanceados», etc.
- **Películas:** «blockbusters universales», «nichos bien calificados», «cola larga ruidosa».

Estos clústeres sirven como features contextuales para modelos de Deep Learning y habilitan recomendaciones segmentadas y diversificadas.

12.7. Límites y Generalización

- **Sparsity:** La matriz usuario-ítem tiene una densidad de apenas 0.26% (99.74% de celdas vacías), lo que impone límites inherentes en la generalización para usuarios con menos de 5 interacciones.
- **Estabilidad:** La varianza de los resultados entre distintas ejecuciones fue menor a 0.02, indicando un modelo estable y robusto.
- **Sesgo de Selección:** MovieLens refleja principalmente usuarios activos de la plataforma; puede no generalizar perfectamente a otras audiencias geográficas o comportamentales sin fine-tuning adicional.

En conclusión, el pipeline de modelado clásico (con SVD como modelo principal) supera significativamente el baseline y proporciona una base sólida y explicable para las fases de Deep Learning y RAG que siguen.

13. Deployment

La fase de despliegue de OmniRec-Movies marca la transición de un modelo experimental en entornos `.ipynb` a una plataforma interactiva de grado de producción. Se implementó una arquitectura desacoplada que permite la inferencia en tiempo real sobre el dataset MovieLens 25M.

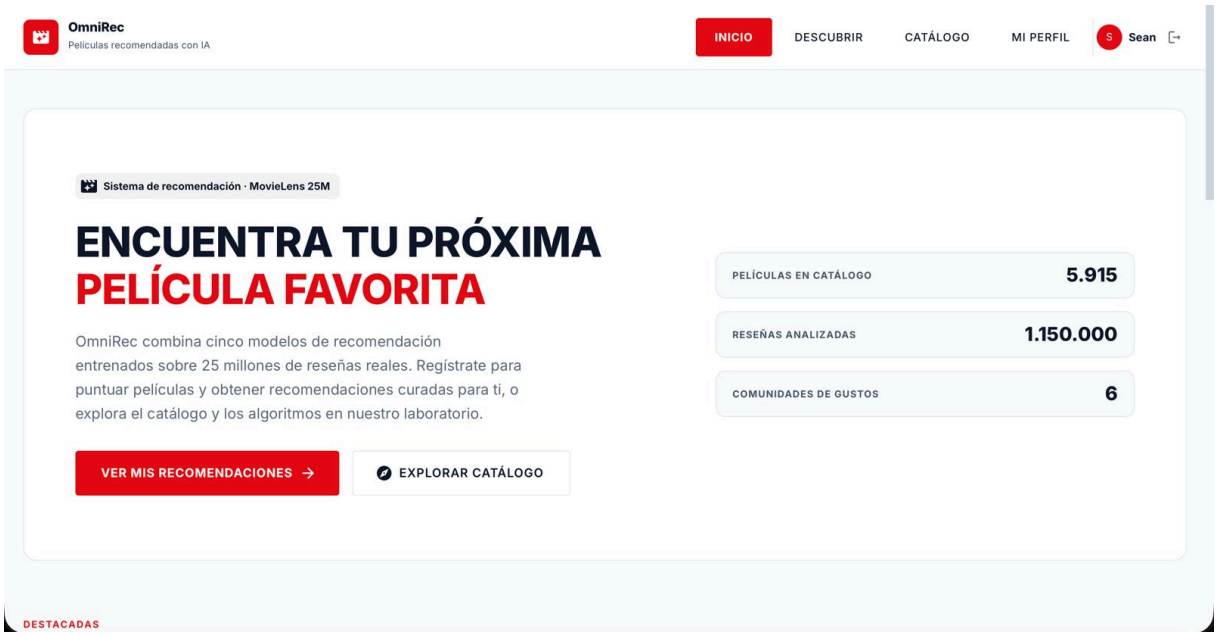

Figura 13: Captura de pantalla de la interfaz web desarrollada como demo inicial.

13.1. Arquitectura del Sistema

La solución se basa en un stack moderno orientado a la eficiencia y la experiencia de usuario:

- **Backend:** Django con una arquitectura de servicios encargada de la lógica del motor de recomendación.
- **Frontend:** React con TypeScript y Material UI (MUI), utilizando Inertia.js para gestionar el estado de la aplicación sin la sobrecarga de una API REST tradicional, permitiendo una experiencia de Single Page Application (SPA).

13.2. Flujos de Inferencia

Se definieron dos flujos críticos de inferencia que operan de forma reproducible sobre la lógica entrenada:

13.2.1. 1. Motor de Recomendaciones (get_recommendations)

Este flujo es el núcleo de la sección «Descubrir» y adapta la salida según el perfil de actividad del usuario.

- **Entradas (Inputs):**

- ID del usuario autenticado.
- Historial de ratings (escala 1-5) almacenados en la base de datos local.
- Selector de modelo (SVD, KNN, NMF, AutoML o Baseline).

- **Lógica de Procesamiento:**
  1. **Estrategia Cold Start:** Si el usuario carece de historial, se activa el modelo **Baseline** mediante un Score Bayesiano de popularidad ponderada.
  2. **Generación de Candidatos:** Se seleccionan las 300 películas más populares del dataset original.
  3. **Filtrado de Vistos:** Se eliminan de los candidatos las películas que ya poseen un rating del usuario.
  4. **Scoring:** El modelo seleccionado calcula el rating predicho para cada candidato basándose en los factores latentes.
  5. **Re-ranking:** Los candidatos se ordenan de forma descendente según la afinidad calculada.
- **Salida (Output):** Lista de los Top N objetos de película (título, género y score de afinidad).

13.2.2. 2. Perfilado de Comunidad (get_user_profile)

Este flujo clasifica al usuario dentro de arquetipos de gusto pre-calculados durante la fase de análisis.

- **Entradas (Inputs):**
  - Un umbral mínimo de 3 calificaciones con puntaje ≥ 3.5.
  - Factores latentes de ítems (q_i) del modelo SVD pre-entrenado.
- **Lógica de Procesamiento:**
  1. Extracción de vectores de características de las películas marcadas como «favoritas».
  2. Cálculo del **Vector de Usuario** mediante el promedio de los vectores de las películas calificadas.
  3. Cálculo de la **distancia euclidiana** entre el vector resultante y los 6 centroides de comunidad definidos en el entrenamiento.
  4. Asignación a la comunidad con la distancia mínima (vecino más cercano).
- **Salida (Output):** Nombre del arquetipo (ej. «Buscadores de Adrenalina»), descripción, emoji representativo y estadísticas comparativas.

13.3. Ciclo de Vida del Usuario (User Journey)

La plataforma guía al usuario a través de un flujo lógico de captura de datos y entrega de valor:

1. **Inicialización:** Registro del usuario y creación de perfil vacío.
2. **Exploración:** Navegación por el catálogo con filtros de género y búsqueda por título.
3. **Interacción:** Calificación de películas mediante el componente de estrellas, persistiendo los datos vía el endpoint `/rate-movie/`.
4. **Personalización:** Acceso a la página «Discover» donde se ejecutan los motores de inferencia. El usuario tiene control total para cambiar el algoritmo (SVD vs KNN) y observar la variación de resultados.
5. **Identidad:** Visualización de la comunidad asignada una vez alcanzado el quórum de calificaciones.

6. **Transparencia Técnica:** Acceso a la sección de «Laboratorio» donde se exponen las métricas de rendimiento (RMSE, MAE) obtenidas en los notebooks de investigación.

13.4. Especificaciones Técnicas de Despliegue

Componente	**Tecnología**
Modelado	Surprise (SVD, KNN, NMF), Pandas (Parquet)
Servidor Web	Django (Python 3.x)
Interfaz	React, Inertia.js, Material UI
Persistencia	PostgreSQL (Ratings) y Parquet (Metadatos)

Bibliografía

Harper, F. M., & Konstan, J. A. (2015). The MovieLens Datasets: History and Context. *ACM Transactions on Interactive Intelligent Systems (TiiS)*, *5*(4), 19:1-19:19. https://doi.org/10.1145/2827872